TMC6460 Qt GUI Project Explanation and STM32 Migration Design

Design document, architecture explanation, register flow, and modular STM32 C interface

Prepared for migration from Qt GUI + Arduino bridge to STM32 application layer

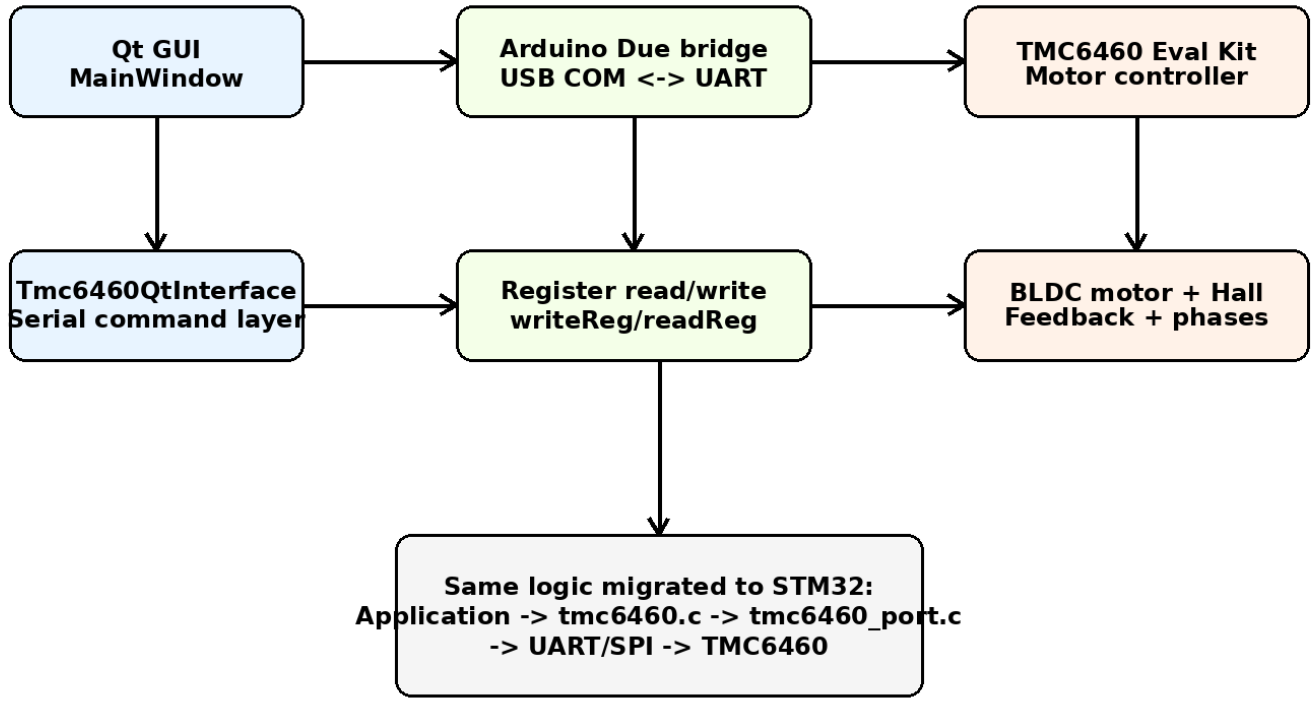
1. Purpose of this document

This document explains the existing TMC6460 Qt project end to end and gives a practical STM32 migration design. The goal is to preserve the working motor behavior from the current Qt/Arduino setup while moving the reusable motor-control logic into generic C files that can be called from an STM32 application layer.

The project has three practical goals: run the BLDC motor in velocity mode, allow torque/current command or limit changes during velocity testing, and continuously monitor status, feedback, and stall/fault conditions.

2. Existing project overview

The current working setup uses a PC Qt GUI to send high-level user commands over a COM port to an Arduino Due. The Arduino Due then communicates with the TMC6460 evaluation kit over UART. The TMC6460 controls the BLDC motor phases and reads Hall/feedback signals.



In the Qt project, the GUI does not directly control PWM or the motor phases. It only presents controls, validates user input, and sends register-level or command-level requests to the lower layer. Actual motor behavior depends on the register configuration written to the TMC6460.

3. Main Qt components

Component	Responsibility	Typical functions/signals
MainWindow	Owns visible widgets and user actions. Updates labels, sliders, buttons, fault indicators, and status text.	Connect, Initialize, Apply Velocity, Apply Torque, E-Stop, periodic refresh timer
Tmc6460QtInterface	Serial communication layer between	openPort(), closePort(), writeReg(),

	Qt and Arduino bridge. Converts GUI requests into commands.	readReg(), setVelocity(), setTorque(), readChipId()
Arduino bridge	Receives PC serial commands and performs UART read/write with the TMC6460.	readRaw(), writeRaw(), readReg(), writeReg(), register dump
TMC6460 register map	Defines motor mode, FOC settings, limits, feedback, and status.	MCC_CONFIG_MOTOR_MOTION, MCC_CONFIG_GDRV, FOC_VELOCITY_TARGET, CHIP_STATUS_FLAGS

4. Qt GUI behavior

4.1 Chip ID display

The GUI reads the chip ID register and formats it for display. For lowercase prefix and uppercase hex digits, use this Qt pattern:

```
QString chipIdHex = QString("%1").arg(chipId, 8, 16, QLatin1Char('0')).toUpper();
chipIdValueLabel->setText("0x" + chipIdHex);
```

This prints values like 0x36343630 while keeping the x in lowercase.

4.2 Velocity controls

The velocity section lets the user enter or slide a raw velocity target. The GUI should keep velocity mode active, write the desired torque target or torque limit when requested, and then write FOC_VELOCITY_TARGET for motion. Slider values should be synchronized with default initialization values so the GUI reflects the actual startup condition.

4.3 Torque controls

Torque mode can remain available for separate testing. For the normal actuator workflow, velocity mode remains the main operating mode, while torque/current command or limit values can be adjusted while the motor is running. This is why the STM32 API includes both TMC6460_SetVelocity() and TMC6460_SetTorqueTarget().

4.4 E-Stop

E-Stop should immediately write velocity target = 0, torque target = 0, and disable the gate driver. After E-Stop, the application should require a deliberate re-enable or re-initialize sequence before accepting new motion commands.

5. Working register configuration summary

The following values are captured from the working velocity-mode path used in the Qt/Arduino tests. They are included in tmc6460_registers.h so the STM32 application does not use hard-coded magic numbers.

Register	Address	Working value	Purpose
CHIP_IO_CONFIG	0x0008	0x70055038	I/O and interface configuration used by the current setup
MCC_ADC_CSA_GAIN	0x00C3	0x00000005	Current-sense amplifier gain setup
MCC_CONFIG_MOTOR_MOTION	0x0100	0x0000E586	Motor motion/control-mode configuration used for velocity operation
MCC_CONFIG_GDRV disable	0x0101	0x80003431	Gate driver disabled state

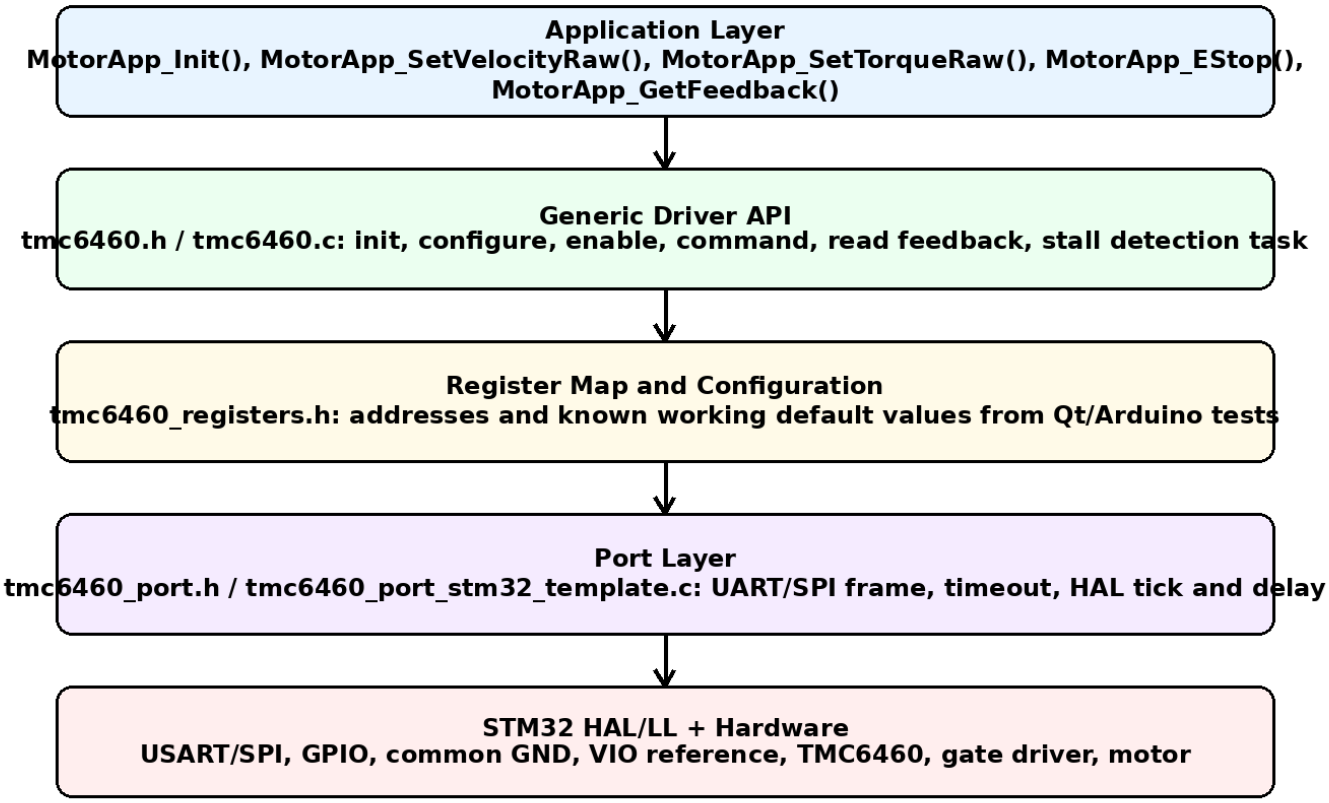
MCC_CONFIG_GDRV enable	0x0101	0x80013431	Gate driver enabled state
MCC_CONFIG_PWM	0x0102	0x00000017	PWM configuration
FOC_PID_CONFIG	0x0140	0x00008558	FOC PID global configuration
FOC_PID_FLUX_COEFF	0x0142	0x01E54C51	Flux PI coefficient
FOC_PID_TORQUE_COEFF	0x0143	0x01E54C51	Torque PI coefficient
FOC_PID_VELOCITY_COEFF	0x0145	0x00640000	Velocity PI coefficient
FOC_UQ_UD_LIMITS	0x0149	0x00001AAA	Voltage vector limits
FOC_TORQUE_FLUX_LIMITS	0x014A	0x13881388	Torque and flux limits
FOC_VELOCITY_LIMITS	0x014B	0x003D0900	Velocity limit
FOC_VELOCITY_TARGET	0x0150	runtime	Velocity target written from GUI/app
MCC_FEEDBACK_CONFIG_CH_A	0x0240	0x052AAAAB	Feedback channel A configuration
MCC_FEEDBACK_CONFIG_CH_B	0x0241	0x06004000	Feedback channel B configuration
MCC_FEEDBACK_CONFIG_VELOCITY	0x0242	0x00001000	Velocity feedback filtering/configuration

6. Recommended initialization sequence

1. Initialize MCU UART/SPI and verify common GND and logic voltage reference.
2. Read CHIP_ID at address 0x0000. Expected value in the existing setup is 0x36343630, which corresponds to ASCII “6460”.
3. Write static configuration registers: I/O config, ADC/CSA gain, motor motion, PWM, FOC PID, limits, feedback channel configuration.
4. Keep gate driver disabled while writing configuration.
5. Read back important registers during bring-up to verify communication and configuration.
6. Enable gate driver only after configuration and safety checks pass.
7. Write velocity target and optional torque target/limit.
8. Run periodic monitor task to read status, actual velocity, actual position, actual torque, and actual flux.

7. STM32 migration architecture

The STM32 migration should not copy the Qt UI structure. Instead, keep a layered firmware structure where the application calls a generic driver API, and the driver calls a port layer for hardware transport. This makes the motor driver reusable across UART, SPI, HAL, LL, or another board.



8. Delivered STM32 C module design

File	Layer	Description
inc/tmc6460.h	Driver API	Application-facing API and handle/config structures
src/tmc6460.c	Generic driver	Initialization, configuration, enable/disable, command writes, feedback reads, stall detection
inc/tmc6460_registers.h	Register map	Addresses, default values, status masks
inc/tmc6460_port.h	Port interface	Hardware abstraction boundary for STM32 UART/SPI
src/tmc6460_port_stm32_template.c	Port implementation template	Place exact UART/SPI register-frame code here
src/tmc6460_app_example.c	Application example	Shows how to call the driver from STM32 application code

9. Application-layer API

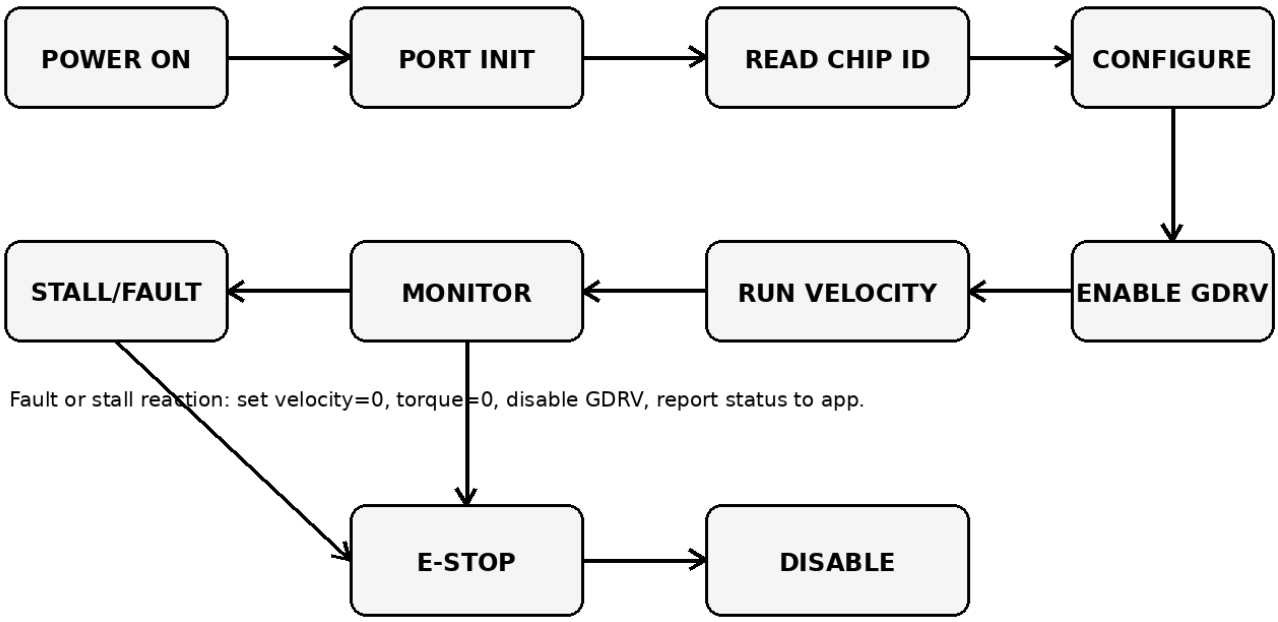
The STM32 application should call only high-level functions. The application should not directly write register addresses except for debug or bring-up.

API	Use
TMC6460_Init()	Initialize port, read chip ID, write default configuration
TMC6460_EnableDriver()	Enable gate driver after configuration

TMC6460_DisableDriver()	Disable gate driver safely
TMC6460_EStop()	Set velocity and torque to zero and disable the driver
TMC6460_SetVelocity()	Keep velocity mode active and write velocity target
TMC6460_SetTorqueTarget()	Write torque target while still allowing velocity workflow
TMC6460_SetTorqueLimit()	Update torque/flux limit register
TMC6460_ReadFeedback()	Read status, velocity actual, position actual, torque actual, flux actual
TMC6460_Task()	Periodic monitoring task, including stall detection

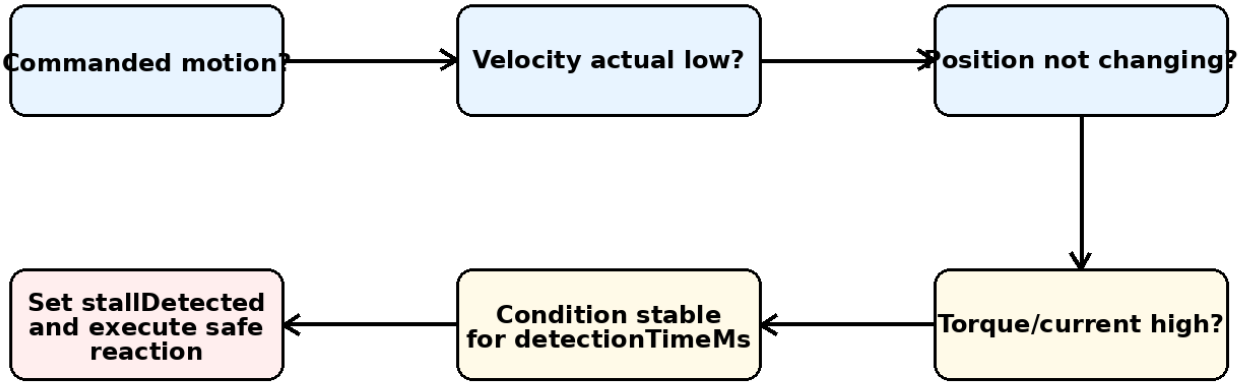
10. State machine

The following state machine is recommended for firmware. It prevents direct motion after reset or E-Stop and makes faults easier to handle.



11. Stall detection design

The software stall detector is based on commanded motion, low actual velocity, position not changing, and high torque/current feedback. A stall is declared only if the condition remains true for detectionTimeMs. To reduce stall detection time, lower detectionTimeMs in the TMC6460_StallConfig_t structure. In the delivered code, the default is 100 ms.



Use this as a software validation layer. Tune minAbsVelocityRaw, maxAbsPositionDeltaRaw, minAbsTorqueRaw, and dete

Parameter	Meaning	Default in delivered code	Tuning guidance
minAbsVelocityRaw	Actual velocity below this is treated as stopped	50	Increase if noisy low-speed readings cause false negative detection
maxAbsPositionDeltaRaw	Maximum allowed position change during stall window	10	Decrease for faster detection; increase if feedback has jitter
minAbsTorqueRaw	Torque/current threshold for load condition	800	Tune using real stall and normal-load logs
detectionTimeMs	How long all stall conditions must remain true	100 ms	Reduce for faster reaction, but verify false trips

12. Port-layer migration note

The delivered tmc6460_port_stm32_template.c intentionally leaves the byte-level TMC6460 register frame as a TODO. This is deliberate because the exact UART frame must match the working Arduino/ADI code. The correct migration path is to copy the proven Arduino readRaw()/writeRaw() framing into the port layer and replace Serial1 write/read with HAL_UART_Transmit() and HAL_UART_Receive().

Once TMC6460_Port_ReadRegister() and TMC6460_Port_WriteRegister() are implemented and CHIP_ID reads correctly, the rest of the driver can be used without changes.

13. STM32 integration checklist

9. Add inc/ and src/ files to the STM32 project include paths and build sources.
10. Enable the correct UART or SPI peripheral in CubeMX or manually in HAL/LL.
11. Connect STM32 TX to TMC6460 UART_RX, STM32 RX to TMC6460 UART_TX, and common GND. Match logic-level reference.
12. Implement TMC6460_Port_ReadRegister() and TMC6460_Port_WriteRegister() using the verified Arduino frame.
13. Call TMC6460_Init() and confirm CHIP_ID = 0x36343630 before enabling the driver.

14. Call TMC6460_EnableDriver() only after register writes pass.
15. Call TMC6460_SetVelocity() from the command layer.
16. Call TMC6460_SetTorqueTarget() or TMC6460_SetTorqueLimit() when torque needs to be adjusted while remaining in velocity operation.
17. Call TMC6460_Task() every 1 ms to 10 ms for feedback refresh and stall detection.
18. Log status flags, actual velocity, actual position, torque actual, and flux actual during first hardware tests.

14. Safety and validation plan

- Start with gate driver disabled and only read CHIP_ID/status registers.
- Use current-limited 24 V supply during initial motion tests.
- Verify motor direction at low velocity before testing higher velocity.
- Validate Hall direction and feedback direction. If feedback sign is wrong, velocity/position loops may vibrate or run away.
- Test E-Stop at low speed before using high speed.
- Record normal-load torque actual values and real stall torque actual values, then tune stall thresholds.
- Do not rely only on a single status flag for mechanical stall. Use feedback-based logic as shown above.

15. Known assumptions and items to verify

- The default register values are taken from the working project logs and should be reviewed against the final ADI register map before production release.
- The port template compiles, but the byte-level UART/SPI frame must be filled from your proven Arduino or ADI transport code.
- Current in mA conversion is not hard-coded because it depends on ADC scaling, current-sense amplifier gain, shunt value, and ADI register interpretation. Keep raw torque/current first, then add calibrated mA conversion after measurement.
- Position target behavior should be verified separately because your working path is currently velocity-first.
- For production, add stronger error handling, retries, watchdog reaction, and fault clear procedure based on final hardware behavior.

16. Quick application example

```
MotorApp_Init();
MotorApp_SetTorqueRaw(300);
MotorApp_SetVelocityRaw(250000);
// Periodic task every 1 ms to 10 ms:
MotorApp_1msOr10msTask();
if (MotorApp_GetFeedback()->stallDetected) { MotorApp_EStop(); }
```

17. Summary

The Qt project is best understood as a PC-based command and visualization layer around a register-configured TMC6460 motor controller. The STM32 migration should not replicate the GUI. It should reuse the same validated register sequence and expose clean modular functions to the application layer. The delivered C files provide that structure: one generic driver, one register map, one port boundary, and one application example.